

Prague University of Economics and Business
Faculty of Informatics and Statistics



**A Data Integration Framework
for Enterprise Application Security**

MASTER'S THESIS

Study program: Applied Data Analytics and Artificial Intelligence

Specialization: Retail and E-Commerce Data Analytics

Author: Bc. Vojtěch Kraus

Supervisor: Ing. Aleš Kotuč

Prague, month YYYY

Acknowledgements

I would like to thank Ing. Aleš Kotuč for his guidance and valuable feedback.

Abstract

To protect their business-critical applications from cybersecurity threats, enterprise security teams rely on numerous tools to detect and resolve vulnerabilities in application source code, configuration, dependencies, CI/CD pipelines, and runtime infrastructure. These tools use different integration patterns, protocols, and data formats, making it difficult to consistently parse, deduplicate, triage, and group their findings. In large environments, additional challenges emerge when linking all findings to the corresponding business applications. This thesis presents a data framework to tackle those challenges, delivering three distinct contributions: (1) a requirements specification for an application security data platform, (2) a reusable and extensible design covering architecture, data model, and medallion-based data pipeline, and (3) a reference implementation built on the Databricks platform. Together, these contributions offer application security teams a foundation for building a robust data platform independent of individual cybersecurity vendors, relieving them of risks usually associated with vendor lock-in.

Keywords

application security, software security, DevSecOps, security integration, data consolidation, medallion architecture, Databricks

Contents

Introduction	11
Motivation	11
Objective	12
Methodology	13
Structure	13
1 Literature Review	14
1.1 Software Development	14
1.1.1 Software Engineering	14
1.1.2 Source Code Management	14
1.1.3 Software Development Lifecycle	15
1.2 Application Security	15
1.2.1 Code Security	15
1.2.2 Software Supply Chain Security	16
1.2.3 Software Lifecycle Security	16
1.2.4 Application Penetration Testing	16
1.2.5 DevSecOps	17
1.2.6 Application Security Data Standards	17
1.3 Data Modeling and Engineering	18
1.3.1 Data Architecture	18
1.3.2 Data Modeling and Pipelines	19
1.4 Related Work and Gap Analysis	19
2 Requirement Analysis	21
2.1 Requirements Methodology	21
2.1.1 Specification Format	21
2.1.2 Requirement Prioritization	21
2.1.3 Test-to-Requirement Linking	21
2.1.4 Traceability Generation	22
2.2 Personas	23
2.2.1 Application Owners	23
2.2.2 Application Developers	23
2.2.3 Security Experts	23
2.2.4 Development and Security Leadership	23
2.3 Data Sources	23
2.3.1 Application Inventory	23
2.3.2 Tools for Software Development and Delivery	23
2.3.3 Runtime Infrastructure Platforms and Security Tools	24
2.3.4 Security Testing Tools	24

2.4	Data Transformations	25
2.4.1	Data Ingestion and Validation	25
2.4.2	Data Normalization	25
2.4.3	Finding Deduplication	25
2.4.4	Business Application Mapping	25
2.4.5	Triage	25
2.5	Data Consumers	25
2.5.1	Remediation Tracking Systems	25
2.5.2	Application Security Posture Management (ASPM)	25
2.5.3	SOAR and SIEM	25
2.6	Non-Functional Requirements	25
2.6.1	Data Quality	25
2.6.2	Extensibility	26
2.6.3	Scalability	26
2.6.4	Performance	26
2.6.5	Security	26
2.6.6	Maintainability	26
2.7	Requirement Specification	26
2.7.1	Data Ingestion	26
2.7.2	Data Processing	26
2.7.3	Data Serving	26
2.7.4	Non-Functional Requirements	26
3	Solution Architecture	27
3.1	Architecture Context	27
3.2	Architecture Principles and Constraints	27
3.3	Technology Selection	27
3.3.1	Data Engineering Platform	27
3.3.2	Pipeline Framework	27
3.3.3	Connectors	28
3.4	High-Level Solution Architecture	29
3.5	Component Design	29
3.5.1	Data Ingestion Tier	29
3.5.2	Data Processing Tier	29
3.5.3	Data Serving Tier	30
3.6	Integration Architecture	30
3.6.1	Connector Strategy	30
3.7	Extensibility Architecture	30
3.8	Security Architecture	31
4	Pipeline and Model Design	32
4.1	Pipeline Overview	32
4.2	Bronze Layer	32
4.2.1	Transformations	32

4.2.2	Schema	33
4.3	Silver Layer	33
4.3.1	Transformations	33
4.3.2	Schema	33
4.4	Gold Layer	34
4.4.1	Transformations	34
4.4.2	Schema	34
5	Implementation	35
5.1	Environment Setup	35
5.2	Connector Implementation	35
5.3	Pipeline Implementation	35
5.4	Pipeline Orchestration	35
5.5	Data Serving	35
5.6	Testing and Validation	35
	Conclusion	36
	Thesis Outcomes and Contributions	36
	Limitations	36
	Future Work	36
	References	37
A	Generative AI Use Disclosure	41
A.1	Text Content	41
A.2	Diagrams and Figures	41
A.3	Source Code	42
A.4	Requirements Specification	42

List of Figures

3.1	Component Design	29
4.1	Medallion Layer Design	32

List of Tables

List of source codes

- 2.1 Requirement Marker Example 22
- 2.2 DLT Expectation Example 22
- 3.1 GitHub Security Connector Implementation 28

Introduction

Motivation

To protect business-critical applications from cybersecurity threats, security teams need to use a large set of application security tools: Static Application Security Testing (SAST) scanners analyze source code for vulnerabilities, Software Composition Analysis (SCA) tools detect issues in software dependencies, Dynamic Application Security Testing (DAST) solutions scan running applications for exploitable weaknesses, and secret scanning tools identify exposed credentials in code.

Further detection mechanisms need to be applied to software integration and deployment pipelines that are prone to their own classes of threats. Increasingly, security teams employ mechanisms that identify problems before they even reach the version control system, either in the form of IDE plugins or as security context tools integrated into AI coding assistants like Github Copilot or Claude Code.

In large enterprises, there may be dozens of such vulnerability detection tools, each producing findings through different Application Programming Interfaces (APIs), data formats, and integration patterns (Anderson, 2020; Stallings & Brown, 2024).

As application security architect in a large organization managing tens of thousands of code repositories and thousands of developers, I have witnessed firsthand how challenging it is to keep such a complex environment secure. Most notably:

- Different detection tools of the same class capture different results as they employ different fundamental scanning paradigms
- The same vulnerability may be reported by multiple tools, requiring careful deduplication
- Findings lack context regarding business criticality of the software project or its threat exposure, both of which are critical determinants of how urgently a vulnerability needs to be remediated
- TODO more

I argue that the challenge of consolidating application security data is, at its core, a data engineering problem. While the industry is addressing this with commercial products categorized as Application Security Posture Management (ASPM) that offer dashboards and aggregated views, these solutions are proprietary, expensive, and inflexible. They introduce vendor lock-in and limit the ability of security teams to customize data pipelines, apply their own machine learning models, or integrate with internal systems on their own terms.

The most notable open-source alternative, DefectDojo, takes a different approach: it is

a Django-based web application designed primarily as a vulnerability management interface. However, it was not built with a data-first architecture in mind. Its scaling capabilities are limited by the constraints of a traditional relational database backend, and its data ingestion model relies predominantly on push-based integrations or manual report uploads. Pull-based connectors that actively fetch data from security tool APIs are available only in the commercial DefectDojo Pro offering, further fragmenting the landscape between open and proprietary solutions.

What is missing is a vendor-agnostic, data-first framework that treats application security data as a first-class data engineering problem. One that provides a reusable architecture for ingestion, normalization, deduplication, and serving of security findings at enterprise scale. This thesis aims to fill that gap.

Objective

The main goal of this thesis is to design and implement a vendor-agnostic data integration framework for enterprise application security. The framework consolidates security findings from heterogeneous tools into a unified data platform, enabling consistent normalization, deduplication, triage, and correlation of vulnerabilities across business applications.

To achieve this goal, I define the following subgoals:

1. **Literature review** ([Chapter 1](#)): Survey existing literature across three domains: software development practices, application security, and data engineering. Identify the theoretical foundations and the gap that motivates this work.
2. **Requirement specification** ([Chapter 2](#)): Analyze the application security domain to identify personas, data sources, required data transformations, data consumers, and non-functional requirements. Produce a requirements specification for the framework, with the full specification included in the appendix.
3. **Architecture** ([Chapter 3](#)): Propose a reusable and extensible architecture that satisfies both analytical and transactional use cases.
4. **Pipeline and model design** ([Chapter 4](#)): Design the data model and processing pipeline following the medallion architecture pattern (Strenghtolt, [2025](#)).
5. **Implementation and validation** ([Chapter 5](#)): Use AI-assisted coding to build a reference implementation on the Databricks platform (Lee et al., [2024](#)) and validate it against the requirements specification through automated testing with requirement traceability.

Methodology

This thesis follows a design-science research methodology as presented by Hevner et al. (2004) and Peffers et al. (2007), structured into three phases.

In the first phase, I conduct a **literature review** of academic and industry sources across three domains: software development, application security, and data engineering. This review establishes the theoretical foundation for the framework and includes vendor documentation for the technologies used in the implementation.

In the second phase, I perform a **requirements analysis and design**. I analyze the application security landscape to identify the key personas, data sources, required transformations, and non-functional requirements. Requirements are prioritized using the MoSCoW method. Based on the requirements, I propose an architecture and design a data model with a medallion-based pipeline.

In the third phase, I build a **reference implementation** on the Databricks platform to demonstrate feasibility of the proposed design. I validate the implementation through automated tests with requirement traceability, mapping individual tests to requirements, and through data quality expectations enforced by Delta Live Tables (DLT).

The thesis delivers three distinct contributions:

1. A **requirements specification** for an application security data platform that other organizations can adapt to their own needs.
2. A **reusable design** covering architecture, data model, and pipeline that is not tied to a specific platform or vendor.
3. A **reference implementation** on Databricks that proves the feasibility of the design and serves as a starting point for production deployments.

Structure

[Chapter 1](#) of the thesis researches relevant literature on software development, application security, and data engineering, and identifies the research gap. [Chapter 2](#) defines the personas, data sources, transformations, consumers, and non-functional requirements, organized in a coherent requirements specification. [Chapter 3](#) presents the platform architecture, including technology selection and component design. [Chapter 4](#) details the medallion-based data model and pipeline transformations across the bronze, silver, and gold layers. [Chapter 5](#) describes the reference implementation on Databricks, covering connector development, pipeline orchestration, and testing. The Conclusion evaluates the outcomes against the objectives defined above, discusses limitations, and suggests directions for future work.

1. Literature Review

This chapter presents the literature across three domains fundamental to this thesis. [Section 1.1](#) reviews software development practices. [Section 1.2](#) examines security testing approaches, tools, and data standards. [Section 1.3](#) covers data architecture and engineering patterns. Finally, [Section 1.4](#) analyzes related work and identifies the gap this thesis addresses.

1.1 Software Development

Before discussing how to secure software, it is necessary to define what modern software development entails. This section surveys the literature on software engineering practices and the Software Development Lifecycle (SDLC).

1.1.1 Software Engineering

Sommerville ([2015](#)) and Pressman and Maxim ([2019](#)) provide comprehensive treatments of the field, covering requirements engineering, design, testing, and project management. At a more theoretical level, Hoffman and Weiss ([2001](#)) collect Parnas’s foundational papers on modularity and information hiding, while Scott ([2015](#)) offers a rigorous treatment of programming language design underpinning modern development tools.

On the practical side, McConnell ([2004](#)) provides guidelines for software construction covering code quality and collaborative practices, Martin ([2008](#)) proposed principles of clean, maintainable code widely adopted in industry, and Hunt and Thomas ([2019](#)) provided a broader pragmatic philosophy for professional development. Oram and Wilson ([2007](#)) present practitioner essays on elegant solutions to real-world problems, illustrating the craft nature of software development. These works agree that software is built collaboratively from source code, organized in repositories, and deployed through increasingly automated pipelines.

1.1.2 Source Code Management

Chacon and Straub ([2014](#)) provides a thorough treatment of Git, covering branching strategies, merging, and distributed workflows, while Skoulikari ([2024](#)) offers a more recent, visually oriented introduction. Winters et al. ([2020](#)) extend this perspective to organizational scale, describing how Google manages a monolithic repository with tens of thousands of contributors. For this thesis, repositories are significant as the primary unit around which

security findings are grouped and tracked, a relationship central to the data model in [Chapter 4](#).

1.1.3 Software Development Lifecycle

SDLC focused literature documents a shift from sequential "waterfall" models to iterative methodologies and DevOps practices. Forsgren et al. ([2018](#)) present empirical evidence that high-performing teams deploy more frequently with shorter lead times, achieved largely via Continuous Integration and Continuous Delivery (CI/CD) automation. Kim et al. ([2021](#)) complement this with practical guidance for implementing DevOps at scale, covering deployment pipelines and feedback loops. The resulting CI/CD pipeline—an automated sequence of commit, build, test, and deploy stages—is directly relevant to this thesis: each stage is an integration point where security tools produce findings that the proposed framework consolidates.

1.2 Application Security

This section reviews the literature across the application security domains relevant to this thesis and concludes with data standards for representing security findings.

Anderson ([2020](#)) provides broad coverage of security engineering, including threat models and software security. Stallings and Brown ([2024](#)) offer a comprehensive textbook on computer security principles. McGraw ([2006](#)) introduces software security activities like code review, architectural risk analysis, or penetration testing that should be embedded throughout the SDLC, while Howard and LeBlanc ([2006](#)) complement this with practical guidelines for defensive coding. Shostack ([2014](#)) addresses threat modeling as a structured approach to identifying security issues early in design. The OWASP Top 10 (OWASP Foundation, [2021](#)) is a widely referenced classification of critical web application security risks that influences how many tools categorize their findings.

1.2.1 Code Security

SAST examines source code, bytecode, or binary code for vulnerabilities without executing the program. Chess and West ([2007](#)) provide the foundational treatment of static analysis for security, covering taint analysis, control flow analysis, and pattern matching to detect injection flaws, buffer overflows, and insecure data handling.

In practice, SAST is implemented by tools such as SonarQube, Checkmarx, Semgrep, and Fortify, which differ in detection approaches: rule-based pattern matching, data-flow analysis, or user-defined custom rules. As noted by Chess and West ([2007](#)), a key challenge is

the tradeoff between false positive rates and detection coverage. From a data integration perspective, each tool produces findings through different APIs and in different formats, a fragmentation that motivates the normalization layer proposed in this thesis.

1.2.2 Software Supply Chain Security

SCA addresses vulnerabilities in third-party dependencies. Modern applications may include hundreds of transitive dependencies, and Anderson (2020) discusses the broader implications of supply chain trust, while Ohm et al. (2020) provide a systematic review of open-source supply chain attacks including typosquatting, dependency confusion, and malicious package injection.

SCA tools such as Snyk, Dependabot, and Mend analyze dependency manifests and cross-reference resolved versions against the National Vulnerability Database (NVD). Several also generate a Software Bill of Materials (SBOM), a structured inventory of all software components, increasingly required by regulatory frameworks (National Telecommunications and Information Administration, 2021). The SBOM concept is formalized through CycloneDX and Software Package Data Exchange (SPDX), discussed further in subsection 1.2.6.

1.2.3 Software Lifecycle Security

The software lifecycle itself introduces security risks. Secret exposure (credentials, API keys, and tokens committed to version control) is detected by tools such as GitLeaks, TruffleHog, and GitHub Secret Scanning, which scan commit history against known credential patterns and entropy-based heuristics.

CI/CD pipeline security is another concern: build systems execute arbitrary code, deployment pipelines hold production credentials, and artifact registries distribute compiled software. The SLSA framework (OpenSSF, 2024) provides progressively stricter requirements for supply chain security, including build provenance and artifact integrity. These lifecycle tools produce findings that differ structurally from code-level vulnerabilities, adding complexity to data consolidation.

1.2.4 Application Penetration Testing

DAST and manual penetration testing examine running systems for exploitable vulnerabilities. Stuttard and Pinto (2011) provide a comprehensive treatment of web application attack techniques, covering injection, authentication bypass, and business logic vulnerabilities, illustrating the attacker’s perspective that informs what security data is most important to capture.

Automated DAST tools such as OWASP ZAP and Burp Suite complement SAST by detecting vulnerabilities that manifest only at runtime. Manual penetration testing produces findings in unstructured formats, typically PDF reports, presenting a distinct integration challenge: while automated tools expose findings through APIs, manual results must be parsed or entered manually.

1.2.5 DevSecOps

The DevSecOps paradigm (Mack, 2023) extends DevOps by embedding security into every phase of the SDLC, commonly called "shifting security left", integrating SAST, SCA, secret scanning, and DAST directly into CI/CD pipelines (Kim et al., 2021).

The practical consequence is a significant increase in both volume and diversity of security data. An organization may operate dozens of tools across pipeline stages, each producing findings in different formats and severity models. McGraw (2006) anticipated this by advocating for security touchpoints spanning the entire lifecycle, but the tooling landscape has grown far beyond what a single touchpoint model can accommodate. This proliferation of fragmented security data is the core problem motivating the framework proposed in this thesis.

1.2.6 Application Security Data Standards

Several standards bring consistency to how security findings are identified, scored, and exchanged.

For identification and scoring, the Common Vulnerabilities and Exposures (CVE) system (MITRE Corporation, 2024) provides a standardized naming scheme for known vulnerabilities. The Common Vulnerability Scoring System (CVSS) (Forum of Incident Response and Security Teams, 2019) scores severity based on attack vector, complexity, and impact, while the Exploit Prediction Scoring System (EPSS) complements it by estimating exploitation probability using Machine Learning (ML) models trained on historical data (Jacobs et al., 2021).

For findings interchange, Static Analysis Results Interchange Format (SARIF) (OASIS Open, 2024) defines a JavaScript Object Notation (JSON)-based format for static analysis results, supported by several SAST tools and GitHub. CycloneDX (OWASP Foundation, 2024) and SPDX (Linux Foundation, 2024) provide schemas for SBOMs and associated vulnerability data. The Open Cybersecurity Schema Framework (OCSF) (OCSF Project, 2024) defines a vendor-agnostic schema for cybersecurity events, though focused on Security Information and Event Management (SIEM) and Security Orchestration, Automation, and Response (SOAR) rather than application security.

No single format covers the full spectrum of application security data. SARIF addresses

static analysis but not SCA or DAST. CycloneDX and SPDX focus on composition, not code-level vulnerabilities. OCSF targets security operations, not application vulnerability management. This gap, the absence of a unified data model across all tool categories, is a central motivation for the data modeling in [Chapter 4](#).

1.3 Data Modeling and Engineering

This section examines data engineering patterns and technologies suitable for consolidating the security data described in [Section 1.2](#).

1.3.1 Data Architecture

The literature documents an evolution from data warehouses through data lakes to the lakehouse paradigm. DAMA International ([2017](#)) provide a comprehensive body of knowledge for data management, establishing the vocabulary and principles referenced throughout this section.

Codd ([1970](#)) established the relational model as the theoretical foundation for data management. The data warehouse, formalized by Inmon ([2005](#)), is a subject-oriented, integrated, time-variant collection of data for analytical decision-making, using a top-down normalized approach. Kimball and Ross ([2013](#)) take a complementary bottom-up approach with dimensional modeling: star schemas organizing data into fact and dimension tables optimized for queries. Both approaches assume structured sources with stable schemas, which does not hold for heterogeneous security tool output.

The data lake (Miloslavskaya & Tolstoy, [2016](#)) stores raw data in its native format, deferring schema definition to consumption (schema-on-read). While this addresses format diversity, it introduces governance challenges; without proper management, data lakes risk becoming “data swamps” (DAMA International, [2017](#)).

The lakehouse architecture (Armbrust et al., [2021](#)) combines data lake flexibility with warehouse governance. Lee et al. ([2024](#)) provide a definitive treatment of Delta Lake, the storage layer adding ACID transactions, schema enforcement, and time travel on top of lake storage. Thalpati ([2024](#)) offer practical guidance for lakehouse design at enterprise scale. The lakehouse paradigm suits the application security problem: it accommodates diverse, semi-structured tool output while providing governance and query performance for both dashboards and operational lookups.

1.3.2 Data Modeling and Pipelines

Kimball and Ross (2013) remain the primary reference for dimensional modeling: fact tables capture measurable events, dimension tables provide descriptive context. Security findings naturally map to facts (each with severity, tool source, and detection date) while repositories, applications, and teams serve as dimensions. DAMA International (2017) also discuss Data Vault, which prioritizes auditability and historical tracking relevant for compliance.

Reis and Housley (2022) cover data engineering fundamentals, including Extract, Transform, Load (ETL) versus Extract, Load, Transform (ELT) patterns. ETL transforms data before loading, requiring upfront schema knowledge. ELT loads raw data first and transforms in place, aligning well with lakehouse architectures where engines such as Apache Spark perform transformations at scale (Lee et al., 2024; Thalpati, 2024).

The medallion architecture (Strengtholt, 2025) organizes lakehouse transformations into three layers:

- **Bronze:** Raw data with minimal transformation, preserving original schemas.
- **Silver:** Cleaned, validated, and normalized data conforming to a unified schema.
- **Gold:** Consumption-ready aggregated metrics and enriched datasets.

Strengtholt (2025) discusses this pattern with Delta Lake and Spark, covering schema evolution and incremental processing. Thalpati (2024) complement with guidance on catalog organization and compute optimization at scale. The medallion architecture is adopted as the core pattern for the pipeline in Chapter 4: bronze ingests raw security data, silver normalizes it into a vendor-agnostic schema, and gold produces metrics for stakeholder consumption.

1.4 Related Work and Gap Analysis

This section examines existing approaches to security data consolidation and identifies the gap this thesis addresses.

Commercial ASPM platforms such as Apiiro, Cyscale, ArmorCode, and Snyk AppRisk aggregate findings into unified dashboards with correlation and risk scoring. However, they are proprietary, operate as black boxes, and create vendor lock-in: organizations cannot customize pipelines, apply their own ML models, or integrate beyond vendor-supported options.

DefectDojo (DefectDojo, 2024) is the most prominent open-source alternative, supporting imports from over 150 tools. However, it was designed as a vulnerability management interface, not a data platform. Its PostgreSQL backend limits enterprise-scale analytics,

and pull-based API connectors are available only in the commercial Pro offering.

Industry data models such as OCSF (OCSF Project, 2024) target security operations (SIEM, SOAR) and do not model application-level entities such as repositories or development teams. Cloud-native security data lakes like AWS Security Lake adopt OCSF for log aggregation but do not address application security consolidation.

Academic literature on application security consolidation as a data engineering problem is limited, with most work focusing on detection techniques rather than integrating findings across heterogeneous tools at scale.

No existing approach satisfies all requirements. Commercial ASPM platforms are proprietary and inflexible. DefectDojo lacks a data-first architecture for enterprise analytics. OCSF targets security operations rather than application security. This thesis fills the gap with an open, vendor-agnostic framework that treats application security consolidation as a data engineering problem.

2. Requirement Analysis

The goal of this chapter is to conduct a requirement analysis and produce a structured requirements specification which we will later use to design the data framework.

2.1 Requirements Methodology

Requirements are gathered through application security domain analysis, drawing from my personal experience, consultations with my professional network, and by having researched literature and industry reports in [Chapter 1](#).

This thesis adopts a test-bound approach to requirements traceability. Rather than maintaining disconnected requirement documents that are prone to becoming stale, implementation status is derived directly from test execution results.

2.1.1 Specification Format

Each requirement is assigned a unique identifier following the pattern REQ-AREA-NNN, where:

- AREA indicates the functional domain (ING for ingestion, PROC for processing, SRV for serving)
- NNN is a sequential number within that domain

2.1.2 Requirement Prioritization

Requirements are documented with priority using the MoSCoW method (Must Have, Should Have, Could Have, Won't Have).

2.1.3 Test-to-Requirement Linking

The framework employs two complementary verification mechanisms aligned with Databricks development patterns.

Unit and Integration Tests

Python modules containing connector logic and transformation functions are tested using pytest with custom requirement markers. [Source code 2.1](#) demonstrates how tests reference requirement IDs.

Source code 2.1

Requirement Marker Example

```
1 @pytest.mark.requirement("REQ-ING-002")
2 def test_sonarqube_pagination():
3     """Validate pagination handling for large result sets."""
4     ...
```

Data Quality Expectations

Data quality requirements are verified using Delta Live Tables expectations, which execute during pipeline runs. [Source code 2.2](#) shows how expectations map to requirements via code comments.

Source code 2.2

DLT Expectation Example

```
1 @dlt.table
2 @dlt.expect_or_fail("valid_severity", "severity IN ('critical', 'high', 'medium', 'low')")
3 @dlt.expect("has_finding_id", "finding_id IS NOT NULL")
4 def silver_findings():
5     # REQ-NFR-001: Data Quality Validation
6     return transform_findings(dlt.read("bronze_raw"))
```

Expectation results are captured in the pipeline event log and reflected in requirement status.

2.1.4 Traceability Generation

A generator script produces the traceability matrix by:

1. Scanning test files for `@pytest.mark.requirement` markers
2. Parsing DLT pipeline code for expectation comments referencing requirement IDs
3. Collecting pytest results and DLT pipeline event logs
4. Merging with requirement descriptions from specification

Appsec vendors, products and their capabilities, APIs and their maturity/completeness

Capture data formats and integration patterns of the tools

2.2 Personas

2.2.1 Application Owners

2.2.2 Application Developers

2.2.3 Security Experts

Architects and Engineers assigned to application teams

Including security compliance / auditors

2.2.4 Development and Security Leadership

Dashboards, metrics, trends, quantified risk

2.3 Data Sources

2.3.1 Application Inventory

Start with looking at things from a business application perspective

Business application inventory + business impact assessment

source systems:

2.3.2 Tools for Software Development and Delivery

important data sources for inventory

start with SCM and issue tracking tools

Github, Gitlab, Bitbucket, Azure DevOps

Jira, Azure DevOps, Github and Gitlab issues

CI/CD pipelines - Jenkins, Github Actions, Azure DevOps

2.3.3 Runtime Infrastructure Platforms and Security Tools

We need infra portrayed from app angle - what does the app run on, how is it secure, whats the security context

AWS - EC2, serverless Azure GCP

Also how secure is it (infra VMDR like Qualys, Wiz...) can impact risk of app vulns if they run on vulnerable infra too

How to get data from Kubernetes, Docker even if on prem

2.3.4 Security Testing Tools

Primary vulnerability discovery tools (secret scanning, , , ...)

Secret Scanning Tools

Static Application Security Testing (SAST) Tools

SAST

Software Composition Analysis (SCA) Tools

SCA

Dynamic Application Security Testing (DAST) Tools

DAST

Penetration Testing

Need a way to upload manual findings, e.g. PDF report from external pentesters

2.4 Data Transformations

2.4.1 Data Ingestion and Validation

Filtering?

2.4.2 Data Normalization

Standard format for each object and finding type

2.4.3 Finding Deduplication

When 2 tools find the same thing, link it, don't create 2 issues

2.4.4 Business Application Mapping

2.4.5 Triage

2.5 Data Consumers

2.5.1 Remediation Tracking Systems

Jira, ServiceNow

2.5.2 Application Security Posture Management (ASPM)

Can pull from data platform already cleaned data instead of from each source separately

2.5.3 SOAR and SIEM

2.6 Non-Functional Requirements

2.6.1 Data Quality

Data validations and transformations on input are critical

2.6.2 Extensibility

Important feature of the whole framework, especially on data source integrations and ML models

2.6.3 Scalability

it's intended for large enterprises, 10k's of repos +

2.6.4 Performance

Some use cases require low latency

2.6.5 Security

Obviously, with this use case

2.6.6 Maintainability

SaaS preferred over manual maintenance - Databricks is ideal

All source code versioned in git and following best practices

2.7 Requirement Specification

2.7.1 Data Ingestion

2.7.2 Data Processing

2.7.3 Data Serving

2.7.4 Non-Functional Requirements

3. Solution Architecture

generic concepts and open industry standards instead of proprietary concepts/technology unless necessary

Needs to support both OLTP/OLAP use cases on data serving tier

3.1 Architecture Context

It will be used to pull data from data sources (those are naturally out of scope)

It needs to be ready to support an application security management dashboard with a combination of OLTP and OLAP use cases (dashboard application also out of scope).

3.2 Architecture Principles and Constraints

3.3 Technology Selection

What I'm choosing and why, what were the alternatives and reasons not to choose them

3.3.1 Data Engineering Platform

Choosing **Databricks** over Snowflake, Fabric, BigQuery

Contains a strong data governance framework - **Unity Catalog**, good for security non-functional requirement

3.3.2 Pipeline Framework

I will use **DLT**, a "data engineering pipeline framework running on top of Delta Lake that combines incremental ingestion, streamlined ETL, and automated data quality processes such as expectations" (Lee et al., [2024](#)).

The advantages of **DLT** are...

3.3.3 Connectors

(general plan, more details in Integration Architecture)

dlt, Fivetran?, Databricks Partners?

data load tool (dlt) is utilized to ingest data from security tools into the Bronze layer. Source code 3.1 shows an example implementation of a connector for GitHub security data.

Source code 3.1

GitHub Security Connector Implementation

```

1  import dlt
2  from dlt.sources.helpers.rest_client import paginate
3
4  @dlt.source
5  def github_security_source(org: str, token: str):
6      """Extract security data from GitHub organization."""
7
8      @dlt.resource(write_disposition="merge", primary_key="id")
9      def repos():
10         """Repository inventory"""
11         yield from paginate(f"/orgs/{org}/repos")
12
13     @dlt.resource(write_disposition="merge", primary_key="number")
14     def code_scanning_alerts():
15         """GitHub Code Scanning (SAST) alerts"""
16         yield from paginate(f"/orgs/{org}/code-scanning/alerts")
17
18     @dlt.resource(write_disposition="merge", primary_key="number")
19     def dependabot_alerts():
20         """Dependabot (SCA) vulnerability alerts"""
21         yield from paginate(f"/orgs/{org}/dependabot/alerts")
22
23     return repos, code_scanning_alerts, dependabot_alerts

```

Unity Catalog

Delta Lake

Python vs Scala vs SQL

3.4 High-Level Solution Architecture

3.5 Component Design

See Figure 3.1

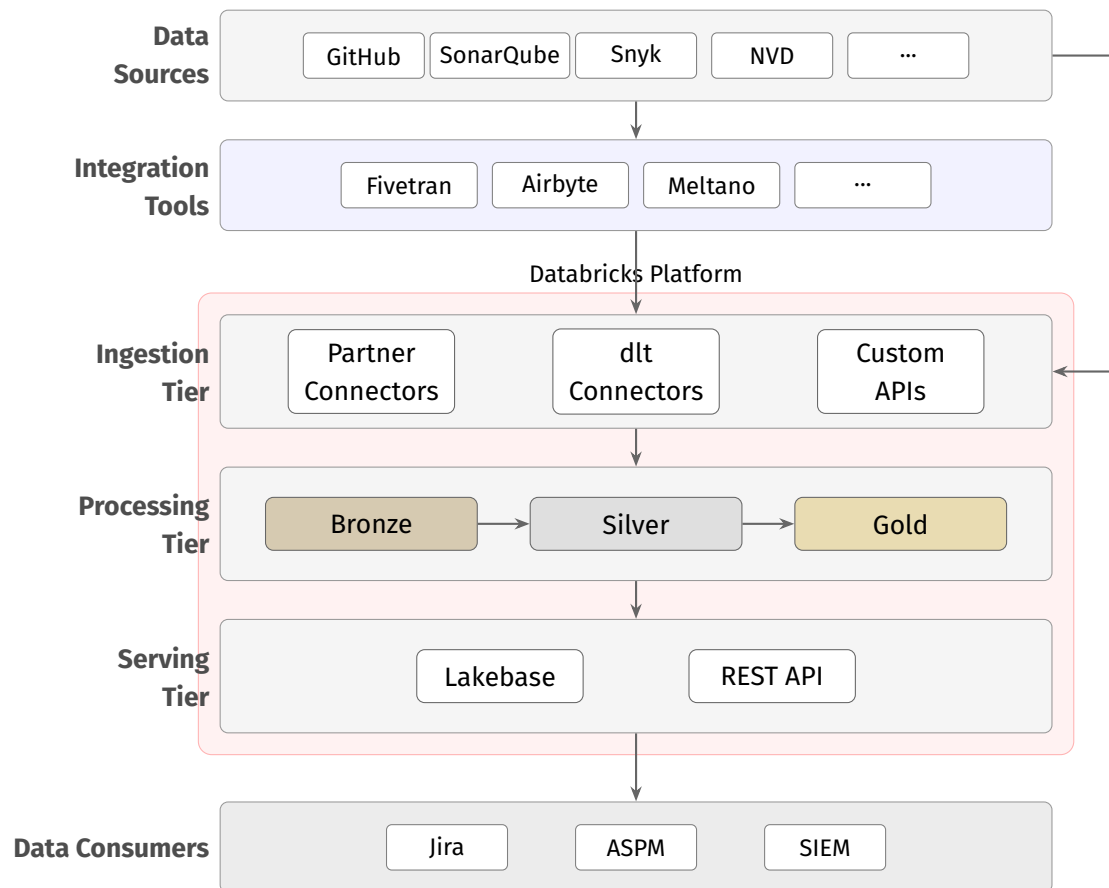


Figure 3.1
Component Design

3.5.1 Data Ingestion Tier

The various connectors

3.5.2 Data Processing Tier

Pipeline Architecture

Medallion architecture, see Chapter 4

DLT for defining transformations

Pipeline Orchestration

Databricks Workflows, don't think my workflows will be complex enough to use Airflow DAGs

3.5.3 Data Serving Tier

Lakebase

3.6 Integration Architecture

3.6.1 Connector Strategy

Different patterns and tools optimal for different data sources

As a service: Fivetran if it satisfies requirements (easiest to do, no need to reinvent the wheel)

Reuse existing: e.g. from DefectDojo <https://github.com/DefectDojo/django-DefectDojo/tree/master/doj>

Databricks Partner

dlt as default tool for custom connectors to REST APIs

Application Inventory

ServiceNow CMDB

Call ServiceNow API vs build ServiceNow app

3.7 Extensibility Architecture

Add new integrations via API

3.8 Security Architecture

4. Pipeline and Model Design

4.1 Pipeline Overview

Each medallion layer is implemented as a schema within a single Unity Catalog.

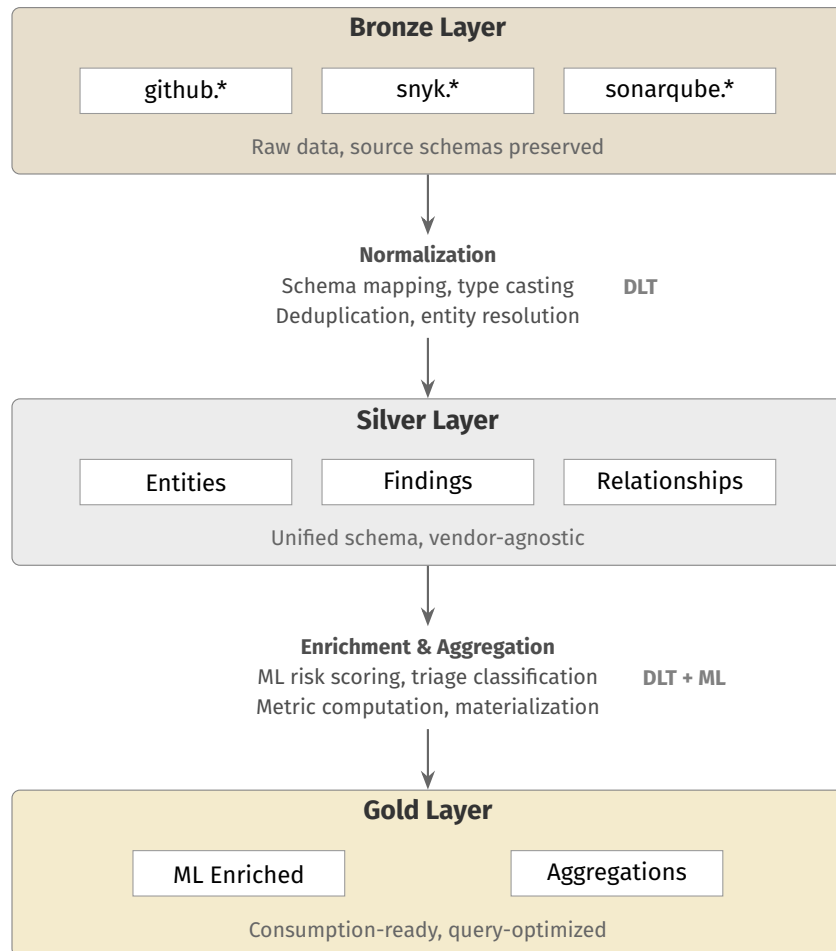


Figure 4.1
Medallion Layer Design

4.2 Bronze Layer

4.2.1 Transformations

Data source request filtering / configuration

Ingestion

4.2.2 Schema

by systems

4.3 Silver Layer

4.3.1 Transformations

Data Quality

Error handling

Normalization

Deduplication

4.3.2 Schema

Vendor-agnostic, unified for each entity and finding

Entities

Implemented as dimensions

Findings

Implemented as facts?

Relationships

(entity-to-entity, finding-to-entity)

4.4 Gold Layer

4.4.1 Transformations

ML Enrichment

Risk scoring

Triage

4.4.2 Schema

Aggregations

Metrics, counts, summaries...

Sliced by severity, app, owner, team, etc (requirement of grouping)

SLAs, trends, MTTR - what matters to stakeholders per stakeholder analysis

Read-Optimized Views

For dashboard queries, materialized, cached

5. Implementation

5.1 Environment Setup

Databricks on AWS

Workspace config

Users, groups, catalogs, schemas

5.2 Connector Implementation

Which connector which pattern and technology

What's the rate limiting on each? This is big data potentially

Authentication, pagination etc, don't forget to mention

5.3 Pipeline Implementation

5.4 Pipeline Orchestration

5.5 Data Serving

5.6 Testing and Validation

Conclusion

This thesis presented a data integration framework for enterprise application security...

Thesis Outcomes and Contributions

Evaluation against objectives from Introduction

Which requirements from Ch2 were met / partially met / not met

What did I deliver extra - ML models beyond requirements?

Limitations

What was not scoped / validated, constraints (license fees for commercial tools that don't give free dev trials)

What was difficult about implementing the design? What could not be implemented in the technology that I selected?

Future Work

More data sources / connectors, based on my provided blueprint

ML model improvements and fine-tuning

Potentials as an open source project

References

- Anderson, R. (2020). *Security engineering: A guide to building dependable distributed systems* (3rd ed.). John Wiley & Sons. (Cit. on pp. 11, 15, 16).
- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR)* (cit. on p. 18).
- Chacon, S., & Straub, B. (2014). *Pro git* (2nd ed.). Apress. (Cit. on p. 14).
- Chess, B., & West, J. (2007). *Secure programming with static analysis*. Addison-Wesley. (Cit. on p. 15).
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387 (cit. on p. 18).
- DAMA International. (2017). *DAMA-DMBOK: Data management body of knowledge* (2nd ed.). Technics Publications. (Cit. on pp. 18, 19).
- DefectDojo. (2024). *DefectDojo documentation*. Retrieved March 6, 2026, from <https://documentation.defectdojo.com/> (cit. on p. 19).
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press. (Cit. on p. 15).
- Forum of Incident Response and Security Teams. (2019). *Common vulnerability scoring system v3.1: Specification document*. Retrieved March 6, 2026, from <https://www.first.org/cvss/v3.1/specification-document> (cit. on p. 17).
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105 (cit. on p. 13).
- Hoffman, D. M., & Weiss, D. M. (2001). *Software fundamentals: Collected papers by David L. Parnas*. Addison-Wesley. (Cit. on p. 14).
- Howard, M., & LeBlanc, D. (2006). *Writing secure code* (2nd ed.). Microsoft Press. (Cit. on p. 15).
- Hunt, A., & Thomas, D. (2019). *The pragmatic programmer: Your journey to mastery* (20th Anniversary). Addison-Wesley. (Cit. on p. 14).
- Inmon, W. H. (2005). *Building the data warehouse* (4th ed.). John Wiley & Sons. (Cit. on p. 18).
- Jacobs, J., Romanosky, S., Adjerid, I., & Baker, W. (2021). Improving vulnerability remediation through better exploit prediction. *Journal of Cybersecurity*, 7(1), tyabo15 (cit. on p. 17).
- Kim, G., Humble, J., Debois, P., Willis, J., & Forsgren, N. (2021). *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations* (2nd ed.). IT Revolution Press. (Cit. on pp. 15, 17).
- Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). John Wiley & Sons. (Cit. on pp. 18, 19).
- Lee, D., Wentling, T., Haines, S., & Babu, P. (2024). *Delta Lake: The definitive guide: Modern data lakehouse architectures with data lakes*. O'Reilly Media. (Cit. on pp. 12, 18, 19, 27).

- Linux Foundation. (2024). *SPDX specification*. Retrieved March 6, 2026, from <https://spdx.dev/specifications/> (cit. on p. 17).
- Mack, S. D. (2023). *The DevSecOps playbook: Deliver continuous security at speed*. John Wiley & Sons. (Cit. on p. 17).
- Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall. (Cit. on p. 14).
- McConnell, S. (2004). *Code complete: A practical handbook of software construction* (2nd ed.). Microsoft Press. (Cit. on p. 14).
- McGraw, G. (2006). *Software security: Building security in*. Addison-Wesley. (Cit. on pp. 15, 17).
- Miloslavskaya, N., & Tolstoy, A. (2016). Big data, fast data and data lake concepts. *Procedia Computer Science*, 88, 300–305 (cit. on p. 18).
- MITRE Corporation. (2024). *CVE program*. Retrieved March 6, 2026, from <https://www.cve.org/> (cit. on p. 17).
- National Telecommunications and Information Administration. (2021). *The minimum elements for a software bill of materials (SBOM)*. Retrieved March 6, 2026, from https://www.ntia.gov/sites/default/files/publications/sbom_minimum_elements_report_0.pdf (cit. on p. 16).
- OASIS Open. (2024). *Static analysis results interchange format (SARIF) version 2.1.0*. Retrieved March 6, 2026, from <https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html> (cit. on p. 17).
- OCSF Project. (2024). *Open cybersecurity schema framework*. Retrieved March 6, 2026, from <https://schema.ocsf.io/> (cit. on pp. 17, 20).
- Ohm, M., Plate, H., Sykosch, A., & Meier, M. (2020). Backstabber’s knife collection: A review of open source software supply chain attacks. *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 23–43 (cit. on p. 16).
- OpenSSF. (2024). *Supply-chain levels for software artifacts (SLSA)*. Retrieved March 6, 2026, from <https://slsa.dev/> (cit. on p. 16).
- Oram, A., & Wilson, G. (2007). *Beautiful code: Leading programmers explain how they think*. O’Reilly Media. (Cit. on p. 14).
- OWASP Foundation. (2021). *OWASP top 10:2021*. Retrieved March 6, 2026, from <https://owasp.org/Top10/> (cit. on p. 15).
- OWASP Foundation. (2024). *CycloneDX bill of materials standard*. Retrieved March 6, 2026, from <https://cyclonedx.org/> (cit. on p. 17).
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77 (cit. on p. 13).
- Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner’s approach* (9th ed.). McGraw-Hill. (Cit. on p. 14).
- Reis, J., & Housley, M. (2022). *Fundamentals of data engineering: Plan and build robust data systems*. O’Reilly Media. (Cit. on p. 19).
- Scott, M. L. (2015). *Programming language pragmatics* (4th ed.). Morgan Kaufmann. (Cit. on p. 14).

- Shostack, A. (2014). *Threat modeling: Designing for security*. John Wiley & Sons. (Cit. on p. 15).
- Skoulíkari, A. (2024). *Learning git: A hands-on and visual guide to the basics of git*. O'Reilly Media. (Cit. on p. 14).
- Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson. (Cit. on p. 14).
- Stallings, W., & Brown, L. (2024). *Computer security: Principles and practice* (5th ed.). Pearson. (Cit. on pp. 11, 15).
- Strenght, P. (2025). *Building medallion architectures: Designing with Delta Lake and Spark*. O'Reilly Media. (Cit. on pp. 12, 19).
- Stuttard, D., & Pinto, M. (2011). *The web application hacker's handbook: Finding and exploiting security flaws* (2nd ed.). John Wiley & Sons. (Cit. on p. 16).
- Thalpati, G. A. (2024). *Practical lakehouse architecture: Designing and implementing modern data platforms at scale*. Apress. (Cit. on pp. 18, 19).
- Winters, T., Manshreck, T., & Wright, H. (2020). *Software engineering at Google: Lessons learned from programming over time*. O'Reilly Media. (Cit. on p. 14).

Appendices

A. Generative AI Use Disclosure

This appendix contains disclosures on all usage of generative AI tools for this thesis.

A.1 Text Content

All textual content in this thesis was drafted and written exclusively by the author in **Overleaf Online LaTeX editor** with logged history of incremental manual edits.

For language revision, Overleaf embedded **AI Assist tool** with **OpenAI GPT** language model was used with following custom prompt:

```
Article usage rules:
- Omit "the" before: plural nouns used generally, abstract concepts,
  technology names, proper nouns, section references
- Omit "a/an" when: introducing concepts already understood from context,
  in technical definitions, after "using/via/through"
- Keep articles only when: specific instance is referenced, first
  introduction of a countable noun, or omission creates ambiguity
- Examples of preferred style:
  - "Bronze layer stores raw data" (not "The Bronze layer stores the raw
    data")
  - "Figure 3 shows architecture" (not "Figure 3 shows the architecture")
  - "Data flows through pipeline" (not "The data flows through the
    pipeline")
```

A.2 Diagrams and Figures

All TikZ diagrams were generated by Anthropic **Claude Opus 4.5** model through iterative prompting. The author provided specifications, reviewed outputs, and iterated until the outcome met structural and visual requirements.

An example prompt for Figure 3.1:

```
Create a TikZ pipeline architecture diagram with vertical flow:

Components:
1. Data Sources row: GitHub, SonarQube, Snyk, NVD, ...
2. External Fivetran box (outside Databricks boundary)
3. Ingestion tier: Fivetran Connector, dlt Connectors, Custom APIs
```

4. Processing tier: Bronze, Silver, Gold layers (colored)
5. Serving tier: Lakebase, REST API
6. Data Consumers row: Jira, ASPM, SIEM

Wrap Ingestion/Processing/Serving in Databricks Platform boundary.
Use arrows showing data flow between tiers.
Style: Soft colors, rounded corners, compact layout.

A.3 Source Code

Implementation code was written by the author with assistance from:

- **Claude Code** (Anthropic) with Opus 4.5 model for code generation, review, and debugging.
- **Databricks AI Assistant** for in-editor code debugging and SQL optimization.

All generated code was reviewed, tested, and modified by the author before inclusion.

A.4 Requirements Specification

Requirements in Chapter ?? follow a test-centric traceability approach. Tests reference requirement IDs via pytest markers, and a generator script produces the traceability matrix by scanning test results. Implementation status is derived from test execution rather than manual tracking.